Noname manuscript No.
(will be inserted by the editor)

SuperSVG++: Semantic-driven Layer-wise Scalable Vector Graphics Synthesis with Semantic Layer Decomposition

Teng Hu[1] · **Ran Yi**[1,†] · **Hongrui Huang**[1] · **Lizhuang Ma**[1] · **Paul L. Rosin**[2] · **Yu-Kun Lai**[2]

Received: date / Accepted: date

Abstract Scalable Vector Graphics (SVG) is a widely used format due to its excellent scalability and editability. However, existing image vectorization methods either suffer from poor reconstruction quality for complex images or require excessive computation time, limiting their practical application. To address these challenges, we propose SuperSVG++, a semantic-driven, layer-wise vectorization model that converts raster images into SVG with high accuracy and efficiency. Firstly, SuperSVG++ proposes a semantic-aware hierarchical vectorization framework, which extracts and reorders semantic layers in a depth-aware manner, ensuring structured and semantic-aware vectorization. Furthermore, we propose a soft-hard boundary constraint in the semantic-aware coarse structure vectorization stage, which improves both the compactness and accuracy of vector paths. To refine details, we propose a difference-conditioned refinement model, which adaptively predicts refining paths based on a difference map computed between the rendered and target image. Additionally, we propose a layer-wise refinement strategy, utilizing a distance-aware priority queue to dynamically decompose and refine semantic regions in a coarse-to-fine and layer-wise manner. Extensive experiments demonstrate that SuperSVG++ outperforms state-of-the-art methods in both reconstruction accuracy and semantic consistency, achieving superior layer-wise vectorization performance.

Keywords Scalable Vector Graphics · Layer-wise Vectorization · Semantic-aware Representation · Semantic Layer Decomposition

† Corresponding author: Ran Yi.
E-mail: ranyi@sjtu.edu.cn
1 Shanghai Jiao Tong University
2 Cardiff University

1 Introduction

Scalable Vector Graphics (SVG) is a widely utilized vector image format with broad applications in web design, graphic design, mobile applications, data visualization, and other fields. Unlike raster images, which represent content through pixels, SVG uses parameterized vectors to define images. This approach provides key benefits, including scalability to any resolution without quality loss and ease of manipulation through its layer-based structure.

Scalable Vector Graphics (SVG) offers strong advantages in image representation and editing, motivating extensive research on image vectorization, which converts raster images into SVG format. Existing methods can be broadly divided into three categories: ***1)*** *Traditional algorithm-based methods* [1,2,3,4,5,6] fit SVG paths to target images using conventional algorithms, but often produce limited vectorization quality. ***2)*** *Deep-learning-based methods* [7,8,9,10,11,12] parameterize and reconstruct raster images with neural networks, achieving efficiency on simple graphics such as icons and emojis, but struggle with complex images. ***3)*** *Optimization-based methods* [13,14,15,16] optimize SVG path parameters for better reconstruction quality, yet require substantial computation time and resources. Overall, existing methods either fail to faithfully reconstruct complex images or suffer from prohibitive computational costs, limiting their practical scalability.

To achieve high-quality and efficient vectorization, our preliminary conference work SuperSVG [17] proposes a superpixel-based coarse-to-fine vectorization model. Since directly vectorizing complex images with neural networks is challenging, SuperSVG decomposes an input image into superpixels with similar colors and textures, and vectorizes each region independently. It

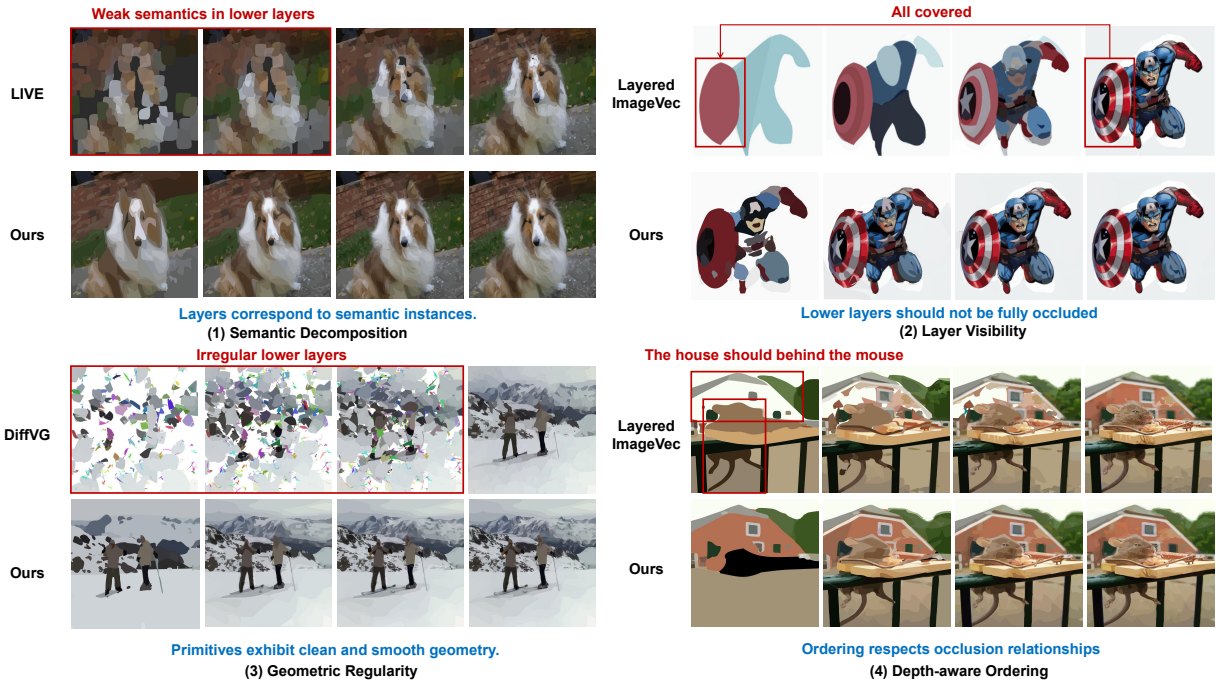


Fig. 1: Problems in the current layer-wise image vectorization methods.

further introduces a Two-Stage Self-Teaching Training Framework, where the coarse-stage model captures the main structure and guides the refinement-stage model to recover fine details. A Dynamic Path Warping loss is also designed to facilitate knowledge transfer between the two stages, enabling fast and accurate SVG reconstruction.

However, SuperSVG is limited to a two-stage coarse-to-refined structure, lacking the semantic layer-wise organization commonly used in human design workflows. Existing layered vectorization methods also remain insufficient for producing high-quality editable SVGs, as they often fail to simultaneously satisfy *Semantic Decomposition*, *Depth-aware Ordering*, *Layer Visibility*, and *Geometric Regularity*, as shown in Fig. 1. Optimization-based methods such as LIVE [14], SGLIVE [18], and Optimize&Reduce [19] mainly rely on global reconstruction losses, producing chaotic and semantically ambiguous primitives. Segmentation-aware methods such as SAM-VG [15], Layered ImgVec [20], and [21] improve semantic awareness but often suffer from poor layer visibility and incorrect ordering due to the lack of explicit depth modeling. Therefore, a new method is needed to inherit the efficiency of SuperSVG while enabling semantic-driven, instance-level layer-wise vectorization.

Building upon SuperSVG [17], we propose **SuperSVG++**, a semantic-driven deep-learning-based vectorization method that achieves accurate and efficient layer-wise raster image vectorization. Unlike SuperSVG, which decomposes an image into superpixels and vectorizes each superpixel independently, SuperSVG++ incorporates semantic information to enable a structured and hierarchical vectorization process. Specifically, SuperSVG++ introduces a **Semantic-driven Vectorization Framework** to construct the semantic structure of SVGs. First, it extracts semantic layers using SegmentAnything (SAM) [22] and DepthAnything [23], and reorders them from far to near to preserve depth-aware composition. To avoid lengthy paths caused by fitting complex semantic boundaries, we further propose a **Soft-hard Boundary Constraint**, which leverages the layered representation to allow paths to cross soft boundaries covered by upper layers, producing more regular and compact paths for downstream SVG editing. To enhance fine details, we propose a **Difference-conditioned Self-refining Model**, which renders an initial vectorized output and uses its difference map from the target image to guide adaptive path refinement. Finally, we introduce a **Layer-wise Refinement Strategy**, where a distance-aware priority queue identifies semantic regions with larger Mean Squared Error (MSE) distances, recursively decomposes them into superpixel regions, and refines them in priority order. This enables SuperSVG++ to improve local details, layer-wise coherence, and overall vectorization accuracy.

Extensive qualitative and quantitative experiments validate the effectiveness of our SuperSVG++. We compare our SuperSVG++ on ImageNet with state-of-the-

art methods, including the commercial software from Adobe Illustrator, showing that SuperSVG++ achieves a much better vectorization accuracy. Moreover, to evaluate the layer-wise effect of the vectorization results, we further introduce a new metric named Layer-wise Score (LS), which demonstrates the best layer-wise vectorization performance of our SuperSVG++.

This paper builds upon our previous conference paper SuperSVG [17], published in the *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024*. In this journal version, we substantially extend SuperSVG into **SuperSVG++**, a semantic-aware layer-wise SVG synthesis framework that improves both vectorization accuracy and SVG editability through semantic layer decomposition, depth-aware ordering, compact path prediction, and adaptive detail refinement. The main new contributions of this journal version are summarized as follows:

- We propose SuperSVG++, a semantic-driven layer-wise vectorization model that converts raster images into SVGs through semantic layer decomposition and layer-wise coarse-to-fine vectorization.
- We propose a soft-hard boundary constraint for semantic-aware coarse structure vectorization, enabling the coarse-stage model to generate compact and accurate paths while preserving editable semantic structures.
- We propose a difference-conditioned path prediction model and a layer-wise refinement strategy, where a distance-aware priority queue dynamically selects regions requiring refinement, enabling adaptive detail enhancement and high vectorization accuracy.

2 Related Work

2.1 Image to SVG

Image vectorization aims to convert raster images into scalable vector graphics (SVG) composed of parameterized vectors. Unlike raster images, which can become blurry when zoomed in, SVGs can be rendered at any resolution without quality loss and are easy to edit, making them popular in web and graphic design. Existing vectorization methods can be categorized into three groups:

Traditional Algorithm-based Image Vectorization Methods can be categorized into mesh-based and curve-based approaches. Mesh-based methods [2,3,4,5] divide an input image into non-overlapping patches and determine the color and boundary location for each region. Curve-based methods [24,25,26,1,27] utilize Bézier curves with distinct colors on either side to construct the vector image. Potrace [1] is a notable example of this type, projecting smooth outlines into Bézier paths and merging adjacent paths. However, the vectorization quality of these methods requires further enhancement.

Deep-regression-based Image Vectorization Methods utilize neural networks to convert raster images into SVG format. Im2Vec [7] uses a variational auto-encoder (VAE) [28] to encode the input image, subsequently mapping it to path parameters through a Long Short-Term Memory (LSTM) module [29]. Raster2Vec [8] targets the vectorization of raster floor plans using a ResNet [30]. Gao et al. [9] employ a pre-trained VGG network [31] and a hierarchical Recurrent Neural Network (RNN) to generate parametric curves of varying sizes. Recently, some works like StarVector [32] and OmniSVG [33] employ large-language models to generate SVG directly, which achieves a better vectorization quality and semantic layer-wise representation. However, these methods are primarily effective for simple images and struggle to accurately vectorize complex images.

Deep-optimization-based Image Vectorization Methods. DiffVG [13] proposes a differentiable renderer that renders the SVG parameters into images. Based on this, DiffVG minimizes the distance between the target raster and vector images by optimizing the SVG parameters using gradient descent. Building upon this foundation, subsequent works introduce layer-wise vectorization to enhance structural representation. Specifically, LayerPeeler [34] leverages Vision-Language Models (VLMs) to decompose images, primarily targeting simple inputs like icons. Focusing on geometric fidelity, methods such as LIVE [14], SGLIVE [18], and Optimize&Reduce [19] design layering algorithms based on graphical features to progressively optimize vector primitives. To further capture high-level semantics, approaches like Layered ImgVec [20], SAM-VG [15], and Less is More [21] incorporate semantic segmentation networks, achieving semantic-aware vectorization. However, despite these advancements, these methods face significant limitations in generating editable and logically ordered vector graphics, as discussed below.

Problems in Current Layer-wise Image Vectorization. To ensure the editability and visual quality of generated vector graphics, we posit that an ideal layer-wise framework must satisfy four essential criteria as shown in Fig. 1: *(1) Semantic Decomposition* into meaningful objects, *(2) Depth-aware Ordering* respecting occlusion relationships, *(3) Layer Visibility* ensuring lower layers remain accessible without redundancy, and *(4) Geometric Regularity* avoiding chaotic artifacts. However, existing methods fail to meet these standards simultaneously. As illustrated in Tab. 1, approaches such

Table 1: **Comparison of properties with existing layer-wise vectorization methods.** We compare SuperSVG++ with the existing layer-wise image vectorization models. Our method is the only one that satisfies all criteria while maintaining high efficiency.

Method	Semantic Decomp.	Layer Visibility	Geometric Regularity	Depth-aware Ordering	High Efficiency
DiffVG [13]	✗	✗	✗	✗	✗
LIVE [14]	✗	✗	✗	✗	✗
SGLIVE [18]	✗	✓	✓	✗	✗
Optimize&Reduce [19]	✗	✓	✓	✗	✗
SAM-VG [15]	✓	✗	✓	✗	✗
Layered ImgVec [20]	✓	✗	✓	✗	✗
Less is More [21]	✓	✓	✓	✗	✗
SuperSVG++ (Ours)	✓	✓	✓	✓	✓

as LIVE [14], SGLIVE [18], and Optimize&Reduce [19] rely primarily on global reconstruction losses; this not only leads to a lack of semantic decomposition but often results in chaotic, irregular shapes in background layers to minimize residuals, violating geometric regularity. Conversely, while methods like SAM-VG [15] and Layered ImgVec [20] incorporate segmentation priors to achieve semantic awareness, they frequently suffer from poor layer visibility, where upper layers completely occlude the underlying ones, rendering the bottom layers redundant and uneditable. Furthermore, the absence of explicit depth information in most existing methods often leads to incorrect layering sequences that contradict spatial reality. To address these challenges, we propose SuperSVG++, which effectively satisfies all four criteria while maintaining high inference efficiency (Fig. 1 (5)).

To achieve efficient and high-quality vectorization, our preliminary conference work SuperSVG [17] proposes a superpixel-based vectorization model that achieves fast and high-precision image to scalable vector graphics (SVG) conversion in a coarse-to-fine manner. However, it lacks semantic layer-wise vectorization effect, where humans usually construct SVG with multiple layers according to its semantics in a coarse-to-fine manner. To solve this problem, we further propose SuperSVG++, a semantic-driven vectorization framework that produces semantically meaningful, layer-wise SVG.

2.2 Text to SVG

Recent research has focused on integrating diffusion models [35] into text-to-SVG generation [36,37,38,39,40], which commonly optimize the geometric and color parameters of vector primitives by leveraging diffusion model priors and employing Score Distillation Sampling (SDS [41])-based loss functions. DiffSkecher [42] and VectorFusion [36] first integrate text-to-image diffusion models [43] to produce vector graphics, showing strong performance in tasks like icon design and sketch generation. To enable layer-wise SVG generation, SVGDreamer [38] proposes a two-stage text-to-SVG generation process, which improves the editability of the individual elements in the generated SVGs. Based on it, SVGDreamer++ [40] further proposes a hierarchical generation technique, enabling better visual quality and higher editability. Different from these text-to-SVG methods, our SuperSVG++ generates SVG from an input image. By combining SuperSVG++ with text-to-image models [43], our method can also generate high-quality images based on text.

3 The SuperSVG Approach

In this section, we review SuperSVG, a deep-learning method that translates images into SVG paths using a two-stage, coarse-to-fine framework, as shown in Fig. 2. To handle complex images, we first decompose the input into perceptually coherent superpixels using the SLIC algorithm [44]. For each superpixel, a coarse-stage model (E_c) generates an initial set of paths to capture the main structure. A refinement model (E_r) then uses the rendered output from the coarse stage to generate additional paths, enriching the details. The entire framework is trained end-to-end leveraging a differentiable renderer [13].

3.1 Superpixel-based Coarse Reconstruction

Coarse-stage Model. Given a superpixel x, E_c predicts a sequence of SVG paths $S = \{s_1, s_2, \cdots, s_n\}$. E_c uses a Vision Transformer (ViT) encoder [45] to extract features, followed by cross-attention and self-attention modules that map these features into learnable path parameters. We train E_c to minimize a combination of three objectives: a normalized pixel-space reconstruction loss ($\mathcal{L}_2$), a boundary loss ($\mathcal{L}_{Bound}$) to restrict paths within the superpixel, and a path-efficiency loss ($\mathcal{L}_{PE}$)

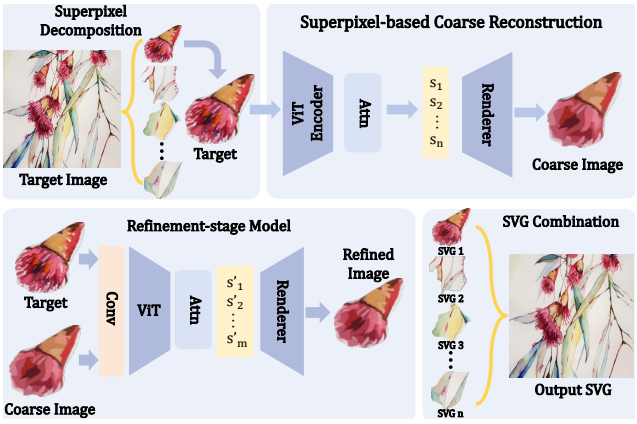


Fig. 2: **Framework of SuperSVG:** SuperSVG decomposes the target image into superpixels, where each superpixel is first vectorized by the coarse-stage model and then the details are refined by the refinement model.

to encourage using the fewest visible paths:

$$E_c^* = \arg\min_{E_c} \mathcal{L}_2 + \lambda_{Bound}\mathcal{L}_{Bound} + \lambda_{PE}\mathcal{L}_{PE}. \quad (1)$$

3.2 Coarse Paths Guided Refinement Stage

Model Architecture. The refinement model E_r addresses the omission of fine details in the coarse stage. It takes the target superpixel x and a canvas $c_1 = R(S)$ rendered from the coarse stage as input. These are fused via convolutional layers and processed by a shared ViT-based architecture to predict additional paths S'.

Coarse Paths Guided Training. Training E_r solely with an $\mathcal{L}_2$ loss leads to a trivial local minimum where the model outputs invisible paths to avoid increasing reconstruction error. We overcome this using a coarse paths guided framework. We split the coarse sequence S into a canvas sequence S_1 and a detail sequence S_2. E_r takes $c_1 = R(S_1)$ as input and uses S_2 as a *pseudo ground truth*. The total objective minimizes error in both the pixel and parameter domains:

$$E_r^* = \arg\min_{E_r} \mathcal{L}_2 + \lambda_{DPW}\mathcal{L}_{DPW} + \lambda_{Bound}\mathcal{L}_{Bound}, \quad (2)$$

where $\mathcal{L}_{DPW}$ measures the distance between the predicted S' and the pseudo ground truth S_2 in the path parameter space.

3.3 Dynamic Path Warping (DPW)

To compute $\mathcal{L}_{DPW}$, standard Dynamic Time Warping (DTW) [46] is suboptimal as it can match multiple target paths to a single generated path, causing the prediction to merely average several targets. To address this, we propose *Dynamic Path Warping (DPW)*, which strictly enforces a one-to-one matching between generated and target paths, allowing some target paths to be skipped for subsequence learning. We compute DPW via dynamic programming and, to ensure differentiability for end-to-end training, substitute the hard minimum operation with the continuous soft-minimum function from SoftDTW [46].

4 SuperSVG++

In this section, we introduce the details of our SuperSVG++, which further improves SuperSVG, mainly aiming to vectorize the raster image in a semantic-aware and layer-wise manner. As shown in Fig. 3, SuperSVG++ contains two stages, including the ***1)*** **Semantic-aware Coarse Structure Vectorization Stage** and ***2)*** the **Layer-wise Refinement Stage**.

In the first stage, SuperSVG++ splits the target image into semantic areas by the Segment-Anything model and reorders them by their estimated depth information. Then, we train a coarse model to fit each semantic area. With the help of the proposed **Soft-hard Boundary Constraint**, we enable the predicted paths to be more compact and more suitable for downstream SVG editing tasks. After the semantic structure is constructed, we employ the difference-conditioned refinement to refine the details. We improve the refinement model with a difference-conditioned module, which takes the difference map between the rendered image and target image as input and outputs the refined path parameters, which effectively enhances the vectorization accuracy. Then, we design a new layer-wise refinement strategy, where we keep a distance-aware priority queue to dynamically determine the areas to be refined, which ensures both accurate vectorization performance and a good layer-wise vectorization process.

4.1 Semantic-Aware Coarse Structure Vectorization

SuperSVG directly decomposes the target raster image into superpixels and then vectorizes each superpixel using the coarse-path prediction model. It can achieve a high vectorization accuracy, but since each superpixel is vectorized independently, all the paths are predicted in a local manner and have little correlation to the semantics, which is different from man-made SVGs and poses a challenge for downstream SVG editing tasks. To solve this problem, SuperSVG++ employs a **semantic-aware** coarse structure vectorization process to enable the model to vectorize the target image according to its semantic information from a global view. Moreover, with

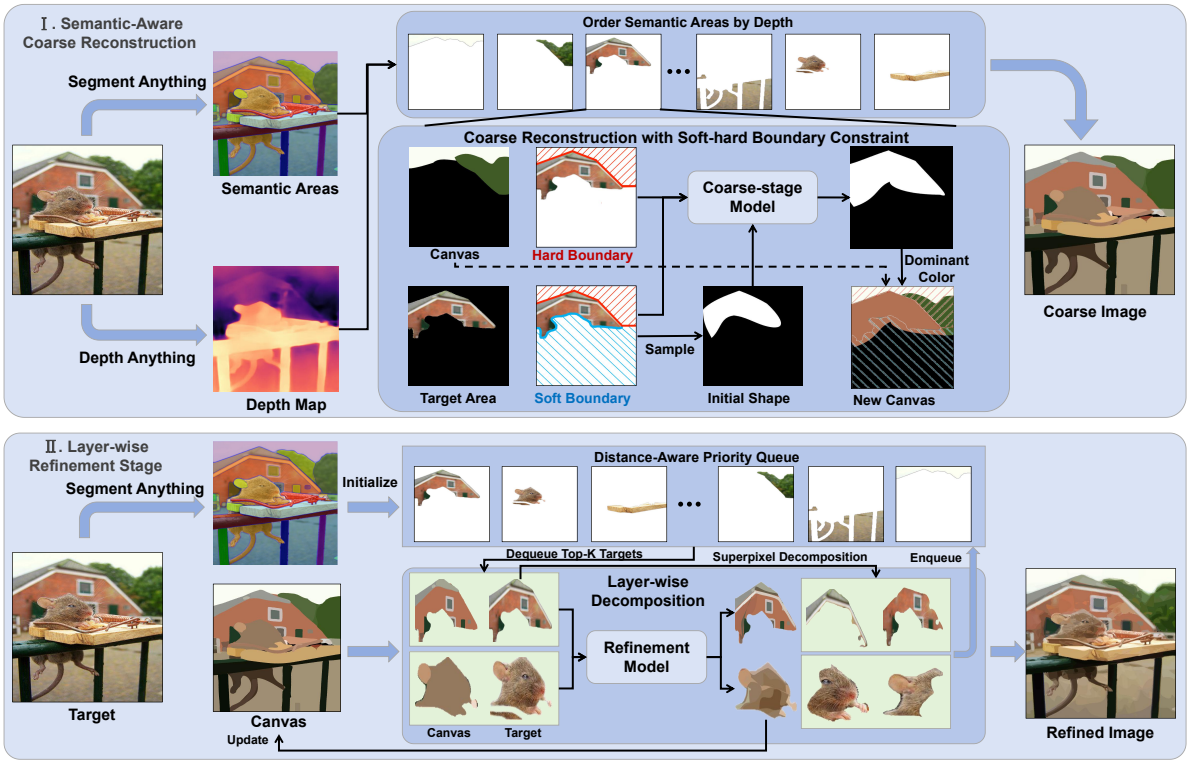


Fig. 3: **Framework of our SuperSVG++:** Our model first introduces a semantic-aware coarse reconstruction stage to reconstruct the semantic structure of the target image based on the soft-hard boundary constraint. Then, we propose a layer-wise refinement stage to refine the details in a layer-wise process.

the help of the proposed **hard and soft boundary constraints**, our model can predict more compact SVG paths that are more suitable for SVG editing.

4.1.1 Depth-guided Semantic Structure Reordering

For an input image $I \in \mathbb{R}^{3\times W\times H}$, we first employ the Segment Anything Model (SAM) [22] to segment the image into several semantic regions $\{M_i\}_{i=1}^{l}$, which include the main semantic information of the input image. Then, to decide the relationship among these semantic regions, we utilize the Depth-Anything Model [23] to predict the depth map $D \in \mathbb{R}^{W\times H}$ of the input image I. With the predicted depth map, we compute the average depth information in each semantic region:

$$D_i = \sum_{x,y}(M_i(x,y)\cdot D(x,y))/\sum_{x,y} M_i(x,y), \tag{3}$$

where (x, y) goes through all the pixel locations of the image.

Afterwards, we reorder the semantic masks $\{M_i\}_{i=1}^{l}$ according to their average depth $\{D_i\}$ in a descending order, i.e., the reordered semantic maps represent the semantic information from the farthest to the nearest. The reordering gives the semantic-level layer-wise vectorization order. For ease of notation, we use $\{M_i\}_{i=1}^{l}$ to represent the reordered semantic masks for the latter sections.

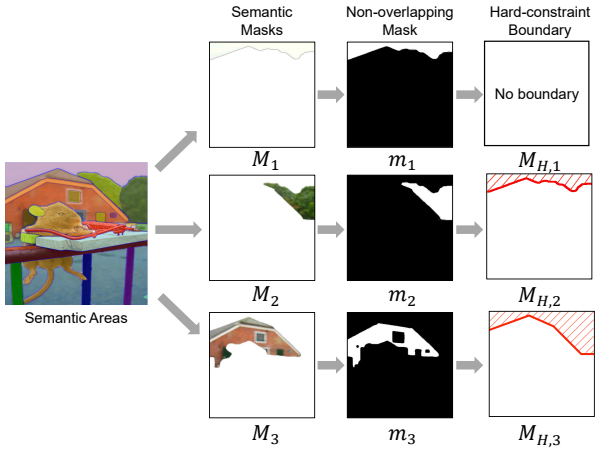


Fig. 4: The process of computing the hard-constraint boundary.

4.1.2 Soft-hard Boundary Constraint

In the SAM mask segmentation, the shape of the semantic area is sometimes too irregular for one path to fit accurately, making it difficult to reconstruct the coarse structure of the target image well. For image vector-

ization, we observe that we do not need to enforce a strict constraint on the original shape directly. Therefore, we propose a **Soft-hard Boundary Constraint**, which can result in a much more compact and accurate vectorization result.

In detail, we define two boundary types during the semantic-structure vectorization stage: **hard-constraint boundary** and **soft-constraint boundary** (shown in Fig. 3 I).

- A hard-constraint boundary is a boundary that the paths cannot cross during optimization;
- A soft-constraint boundary is a boundary that can be crossed by the paths during optimization.

After the depth-guided semantic region reordering process, we have a semantic mask sequence $\{M_i\}_{i=1}^{m}$. Since some of the masks are overlaid together, we first compute a *non-overlapping mask* m_i for each semantic region M_i, where m_i indicates the region inside M_i that is not covered by the subsequent masks $\{M_i\}_{j>i}$. Then, as shown in Fig. 4, for each semantic region M_i, we enumerate the boundaries of the non-overlapping masks in the layers lower than M_i, *i.e.,* $\{m_j\}_{j<i}$, and combine them together as the **hard-constraint boundary** (that cannot be crossed by paths). Meanwhile, the **soft-constraint boundary** is set as the boundary of the semantic region M_i itself (except for the hard-constraint parts). In this way, we relax the constraint of each path, *i.e.,* each path does not need to strictly fit the complex shape of the target semantic mask, but could instead cross the soft-constraint boundary a bit to have a more regular shape. With our strategy, each path will cover the current semantic mask M_i, which can exceed the soft-constraint boundary, while avoiding crossing the hard-constraint boundary. As a result, the shape of the paths can be more compact, and it is much easier to reconstruct the target image.

With the soft-hard boundary constraint, the goal of our coarse model is to predict the path parameter for one SVG path S_i to fit the semantic area M_i while preventing it from crossing the hard-constraint boundary $M_{H,i}$. To achieve this goal, we define an enclosed region for the hard-constraint boundary $M_{H,i} = \cup\{m_j\}_{j<i}$ as the union of the non-overlapping masks in the layers lower than M_i (Fig. 4 red shaded area), which indicates the region that the paths are not allowed to cover, and propose a new loss function:

$$\mathcal{L}_B = \mathop{\mathbb{E}}_{p\sim mask} |R(S_i) - M_i| \cdot M_i + R(S_i) \cdot M_{H,i}, \tag{4}$$

where the first term $|R(S_i) - M_i| \cdot M_i$ computes the area within the current semantic region M_i that is not covered by path, which encourages the predicted path to cover the current semantic region as much as possible.

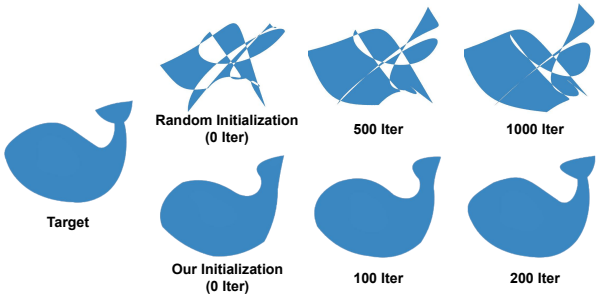


Fig. 5: Comparison of random path initialization and our path initialization methods. When fitting the target shape, random initialization has a severe self-intersection problem, which causes the shape to be too irregular and difficult to fit the target shape. With our initialization method, the path fits the shape soon while keeping a much more compact shape.

Then, the second term $R(S_i) \cdot M_{H,i}$ computes the area of the paths that are outside the hard-constraint boundary, which discourages the predicted path from crossing the hard-constraint boundary. Therefore, with this loss term, the coarse model tends to fit the target semantic area while avoiding crossing the hard-constraint boundary.

4.1.3 Semantic-aware Coarse-path Prediction Model

To vectorize the semantic structure for the input image I, we propose a semantic-aware coarse-path prediction model E_c, which takes a semantic region of the original image $I \cdot M_i$ as input, and predicts one SVG path $s_i = E_c(x)$ to reconstruct this region. E_c follows the main structure of the coarse model in SuperSVG (Sec. 3.1), consisting of a ViT encoder and a cross-attention module followed by a self-attention module.

However, we found that directly predicting a path to fit semantic region with random initialization may cause the shape of the predicted path to be irregular, resulting in some self-intersection problems as shown in Fig. 5. This is because if the control points of the path are randomly initialized at the beginning of the training process, the disordered control points cannot be corrected when training the model to fit the target shape, which leads to self-intersection problems in the final predicted path.

Path Initialization. To solve this problem, we improve the model architecture for the semantic-aware coarse-path prediction model. Specifically, for the semantic regions with only one connected component, we calculate the initial control points by sampling them uniformly along the boundary of the semantic mask. For the semantic regions with several connected components, we sample the control points along the boundary of the bounding box (minimum rectangle) that can cover

all connected regions. Then, we initialize the model by replacing the path queries T_q in the cross-attention module with the initial path parameters $s_{i,0}$, and then the coarse model is designed to predict the residual $\Delta s_i = E_c(x, s_{i,0})$ of the initial path parameters, *i.e.*, the final predicted path parameters are $s_i = s_{i,0} + \Delta s_i$. To avoid the model disturbing the relationship of the initial control points at the beginning of the training process, we introduce a zero-initialization to initialize the final linear projection layer by 0 weights.

Dominant Color for Semantic Regions. We design the semantic-aware coarse model E_c to only reconstruct the shape of the target semantic region, with more texture details added by the refinement model E_r. To this end, we assign a color to each semantic region. If we assign the average color inside the target mask for each path, we will get a lower MSE distance for the coarse results. However, considering the coarse-stage model only aims to reconstruct the semantic layers of the target image, and the refinement model will enrich the details, it is not the best solution to assign the path with the average color inside the corresponding semantic areas. We define the dominant color inside each mask as the color that appears in most of the pixels. If we assign a dominant color for each path, the refinement model does not need to refine the areas with the dominant color, which largely simplifies the job of the refinement model. Therefore, we employ the dominant color to render each predicted coarse path. Details of dominant color calculation are provided in the supplementary material.

4.2 Layer-wise Refinement Stage

In the semantic-aware coarse structure reconstruction phase, our coarse-stage model E_c predicts an SVG that captures and reconstructs the primary semantic structure of the input image I. The rendered image from the SVG, denoted as I_c, generally resembles I, but lacks finer details. To enhance the image details, we utilize a refinement model E_r to predict additional SVG paths, thereby adding more details based on the current canvas c_1. To enable the model with a more accurate refining ability, we propose a new **Difference-Conditioned Self-Refining Model Structure**, which consists of a path prediction module and a difference-conditioned path refining module. Finally, we introduce a **Layer-wise Refinement Strategy** based on dynamic superpixel decomposition, which further improves the accuracy and layer-wise effect in the refinement stage.

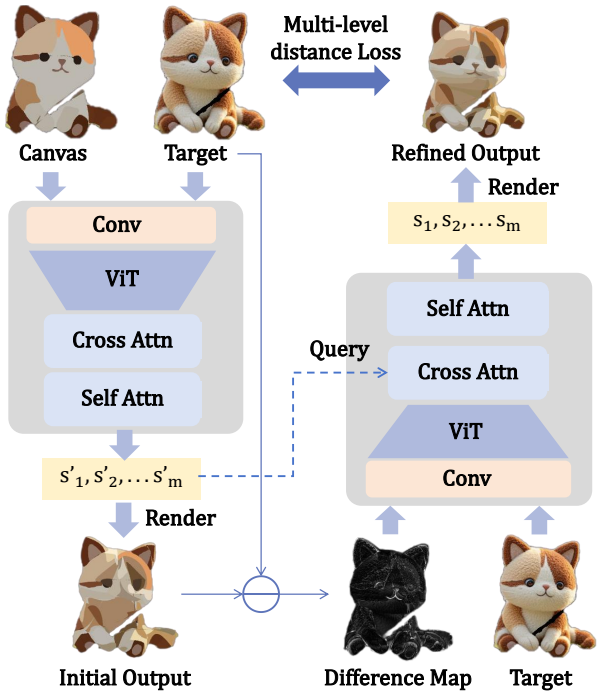


Fig. 6: Difference-conditioned refinement model.

4.2.1 Difference-conditioned Self-refining Model Structure

In SuperSVG, we designed a coarse path guided refinement model (Sec. 3.2), which takes a canvas c_1 and the target superpixel I as inputs and outputs the refinement paths to enrich the details of the canvas. Our previous refinement model consists of a ViT encoder and two attention modules (one cross-attention module and one self-attention module). In SuperSVG, we have found that as the number of self-attention modules increases, the model performance does not improve. This is due to the limited information captured by the ViT features. As the depth of the attention modules increases, less information is kept. Therefore, directly increasing the number of attention layers cannot improve the model performance.

Difference-conditioned Path Refinement Module. To solve this problem, based on the refinement model in SuperSVG (denoted as $E_{r,0}$), we further introduce a Difference-conditioned Path Refinement module $E_{r,1}$ in our refinement stage, *i.e.*, after an initial refined output is predicted, we compute a difference map between the output and the target image, and then input this difference map as a condition to guide further refinement. In this way, our refinement model consists of two parts (Fig. 6): a path prediction module denoted as $E_{r,0}$, and a path refinement module denoted as $E_{r,1}$. 1) The path prediction module $E_{r,0}$ inherits the structure

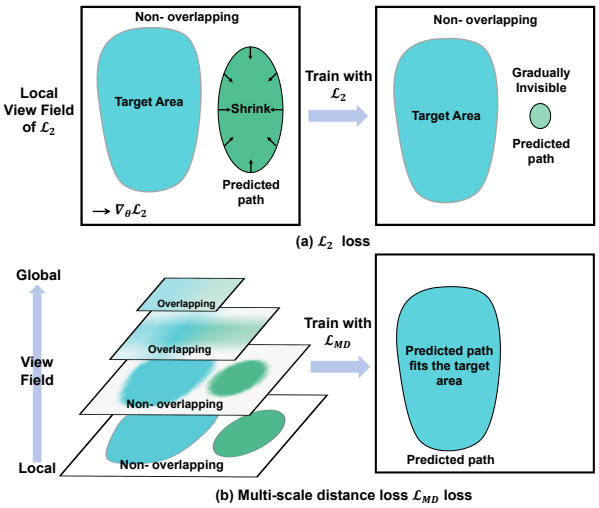


Fig. 7: Comparison between L_2 loss and multi-scale distance loss $\mathcal{L}_{MD}$ when fitting the target area with the predicted path. When the initial predicted path does not overlap with the target region, the local field of view of L_2 loss causes the predicted path to gradually shrink (see Sec. 4.2.1 for further analysis). In comparison, the new multi-scale distance loss $\mathcal{L}_{MD}$ has different fields of view at different scales, where the two areas overlap when the downsample scale factor is large, therefore guiding the predicted path to fit the target area well.

and weights of the refinement model in SuperSVG. It inputs the target image I and current canvas c, and predicts initial paths S_0 which can refine the details for the canvas. We render the predicted paths on the canvas to get the initial refined image y_0. 2) Then, for the difference-conditioned refinement, we compute the difference map DM between the target image I and refined image y_0 by $DM = x - y_0$, which is further input into the path refinement module $E_{r,1}$. The path refinement module $E_{r,1}$ is composed of a ViT encoder, a cross-attention module, and several self-attention modules. The ViT encoder first encodes the difference map into image features, then the cross-attention module projects the image feature into Key and Value, and projects the initial paths S_0 into the Query. Then, it computes the correlation between the difference map and the initial paths. With the following self-attention modules, the refinement module $E_{r,1}$ finally outputs the paths which can largely improve the reconstruction accuracy.

Multi-scale Distance Loss. As shown in Fig. 7 (a), when the initially predicted path s does not overlap with the target region, and we aim to optimize the predicted path to fit the target region with the $\mathcal{L}_2$ loss ($\mathcal{L}_2$ distance between the rendered image from s and the target image), the predicted path s gradually disappears. This is because, when the predicted path does not overlap with the target region, since the renderer only considers the gradient of the region inside the predicted path, the gradient in the target region cannot be backpropagated to the path parameters. I.e., only when the initial predicted path overlaps with the target region, the path parameters can be optimized with $\mathcal{L}_2$ loss alone.

To solve this problem, we propose a multi-scale distance loss (as shown in Fig. 7 (b)), which can enrich the model with a global field of view. Specifically, for two images I (the target image) and $\hat{I}$ (the rendered image of predicted paths) of size $W \times H$, we repeatedly downsample them with scale factor k and compute the MSE distance between the downsampled images for each scale, which can be formulated as:

$$\mathcal{L}_{MD} = \sum_{i=0}^{\log_k \min(H,W)} w_i \| Down_k^i(I) - Down_k^i(\hat{I}) \|, \quad (5)$$

where $Down_k^i(\cdot)$ is the downsample operation with downsample factor k^i, and w_i is the weight for the MSE distance at scale i.

With the multi-scale distance loss $\mathcal{L}_{MD}$, even when the predicted path does not overlap with the target area, as the downsample factor increases, the downsampled images of the predicted path and the target area will overlap with each other. Then, the valid gradients in the overlapping region can guide the predicted path to move to and gradually fit the target area, thereby achieving a good vectorization performance.

4.2.2 Layer-wise Refinement Strategy

The previous SuperSVG output only contains two layers, one for the coarse stage and one for the refinement stage, which leaves room for improving the layer-wise effect, since human artists usually create an SVG with multiple layers in a coarse-to-fine manner, where each layer improves the details of the previous ones. Therefore, to improve the layer-wise effect of our SuperSVG++, we propose a **layer-wise refinement strategy**, which dynamically decomposes the semantics into different layers of superpixels.

Specifically, we keep a distance-aware priority queue $\mathcal{H}$. In the beginning, for each semantic region M_i, we compute the sum of the MSE distance between the rendered image and the target image inside the region. Then, we enqueue these semantic masks into the priority queue $\mathcal{H}$ with the MSE distance as their values. Therefore, the priority queue $\mathcal{H}$ can automatically dequeue the semantic masks with the highest MSE distances, which is the most urgent region that needs to be refined.

We decompose the refinement stage into K layers, for each layer, we first dequeue the Top-m masks in $\mathcal{H}$

and refine them by our difference-conditioned refinement model. Then we decompose the selected masks into n superpixels and compute their MSE distance, and we also enqueue these superpixels regions into the priority queue $\mathcal{H}$. In this way, in later layers, the superpixels with higher MSE distances will be refined first. In this manner, our model can adaptively choose the most urgent superpixel to decrease the final MSE distance, and the layer-wise decomposition method can ensure a good layer-wise effect, thereby improving final vectorization performance in both accuracy and layer-wise effect.

4.3 Integration with Semantic Layering Models

SuperSVG++ can also be extended by incorporating recent semantic-aware image layering models, such as Qwen-Image-Layered [47]. Given an input raster image, Qwen-Image-Layered decomposes it into several semantically disentangled RGBA layers, where each layer contains an alpha map and the corresponding completed RGB appearance. These generated layers provide useful semantic priors for our vectorization process, including cleaner object-level regions, a more explicit layer-wise organization, and more complete object content for partially occluded regions.

In this configuration, we first apply Qwen-Image-Layered to obtain the layered RGBA representation of the input image. For each generated layer, we use its alpha map as the semantic mask and take its completed RGB content as the layer-wise vectorization target. We then vectorize these layers sequentially following their layer order. Specifically, the coarse vectorization model predicts compact paths for each semantic layer based on the corresponding alpha mask, while the dominant color of each layer is used to initialize the coarse SVG appearance. After the coarse semantic layers are constructed, the difference-conditioned layer-wise refinement stage is applied to further recover local geometric structures and color details.

Compared with the default SAM/depth-based semantic construction process, this semantic-layering configuration can provide more explicit and complete layer targets, which is beneficial for producing semantically organized and editable SVGs. However, since Qwen-Image-Layered introduces an additional generative layer decomposition step, it also increases the overall inference time and computational cost. Therefore, we regard it as a high-quality configuration of SuperSVG++ for scenarios where better semantic layer decomposition is preferred over faster processing.

5 Experiments

Implementation Details. We use SVG paths composed by cubic Bézier curves as the vector primitive, where each SVG path is closed, composed of 4 cubic Bézier curves connected end-to-end and has a fill color. Each SVG path has 28 parameters (24 for shape, 3 for color, and 1 for visibility). The coarse-stage model is trained to predict 128 paths for each superpixel first. Then, the refinement model is trained to predict 8 paths at one time. We train both the coarse-stage model and refinement model on the ImageNet dataset [48]. We set batch size as 64 and learning rate as 2.5×10^{-4}. We train the coarse-stage model for 200K iterations with 5K warm-up iterations, and train the refinement model for 200K iterations with λ_{DPW} uniformly decreasing from 1×10^{-3} to 0 over 10K iterations. In the following experiments, we implement our model with two versions: **1) SuperSVG++,** that first predicts coarse semantic SVG paths in a semantic-driven global view and then enriches the details in a layer-wise decomposition process. and **2) SuperSVG++*,** which further optimizes the SVG parameters from SuperSVG++ with $\mathcal{L}_2$ loss, and takes around 10 seconds for optimization. All experiments are carried out on an NVIDIA GeForce RTX 4090 24GB GPU.

5.1 Image to SVG Comparison

State-of-the-art methods. We evaluate our proposed method against a comprehensive set of state-of-the-art baselines covering traditional, deep-learning-based, and optimization-based approaches. For traditional algorithm-based vectorization, we compare with **Potrace** [1] (utilizing Color Trace for color quantization), VTracer [49], and the widely-used commercial software **Adobe Illustrator** [27]. Regarding deep-learning-based approaches, we include **Im2Vec** [7], as well as recent large-model-based methods, including **StarVector** [32] and **OmniSVG** [33], which directly generate SVG primitives. Furthermore, we extensively compare with differentiable optimization-based methods, including the foundational **DiffVG** [13], the topology-aware **LIVE** [14], the geometric-constraint-enhanced **SGLIVE** [18], the path-simplification-focused **Optimize&Reduce** [19], and the semantic-prior-guided **Layered ImgVec** [20]. We utilize the official implementations and settings for all competing methods to ensure a fair comparison.
Quantitative Comparison. We further conduct a quantitative comparison on 1,000 images sampled from the ImageNet [48] dataset and 300 images from MMSVG-Bench [33]. Note that for LIVE, we only evaluate on 50 images due to its extremely long optimization time



Fig. 8: Qualitative comparison with the state-of-the-art methods under different numbers of SVG paths.

(e.g., over 5 hours for one image with 500 paths). The quantitative results are presented in Tab. 2 and Tab. 3. Our SuperSVG++* (with 10 seconds of optimization) consistently achieves the state-of-the-art performance across all metrics (MSE, PSNR, LPIPS, and SSIM) and path numbers, significantly outperforming the second-best method, Layered ImgVec, while being orders of magnitude faster (10s vs. hundreds to thousands of seconds). Notably, this superiority is consistent across both complex real-world natural images from ImageNet and SVG-style graphic images from MMSVGBench, demonstrating that our method is not limited to natural-image vectorization but also generalizes well to practical SVG-style scenarios. Even without extensive optimization, our standard SuperSVG++ maintains a superior balance between quality and efficiency. It surpasses most baselines (e.g., DiffVG, Adobe, SGLIVE, and the recent StarVector) in reconstruction quality. On MMSVG-Bench, our method also achieves the best overall performance, showing strong robustness on icon-like graphics, flat illustrations, and other SVG-oriented images. Although Layered ImgVec achieves competitive accuracy, it incurs a prohibitive computational cost (e.g., 237.10s vs. 2.53s for 500 paths). In contrast, our method delivers high-fidelity vectorization in real-time.

Qualitative Comparison. We further conduct a quantitative comparison on 1,000 images randomly sampled from ImageNet [48] dataset (50 images for LIVE due to the extremely long optimization time: each input image takes about 6 GPU hours to optimize under 500 SVG paths). The quantitative results are presented in Tab. 2. Our SuperSVG++ achieves the best image vectorization results across different numbers of SVG paths. With 10 seconds of optimization, SuperSVG++* significantly improves vectorization accuracy, outperforming the compared methods by a large margin.

Generalization to SVG-style graphic scenarios. To further evaluate the generalization ability of different vectorization methods beyond natural images, we additionally conduct qualitative comparisons on SVG-style graphic scenarios. Specifically, we select representative simple icons from MMSVGBench, and collect SVG-style characters, flowcharts, and posters, which are closer to practical vectorization applications than ImageNet images. We compare our SuperSVG++ with the representative methods, including Layered ImgVec [20], Om-

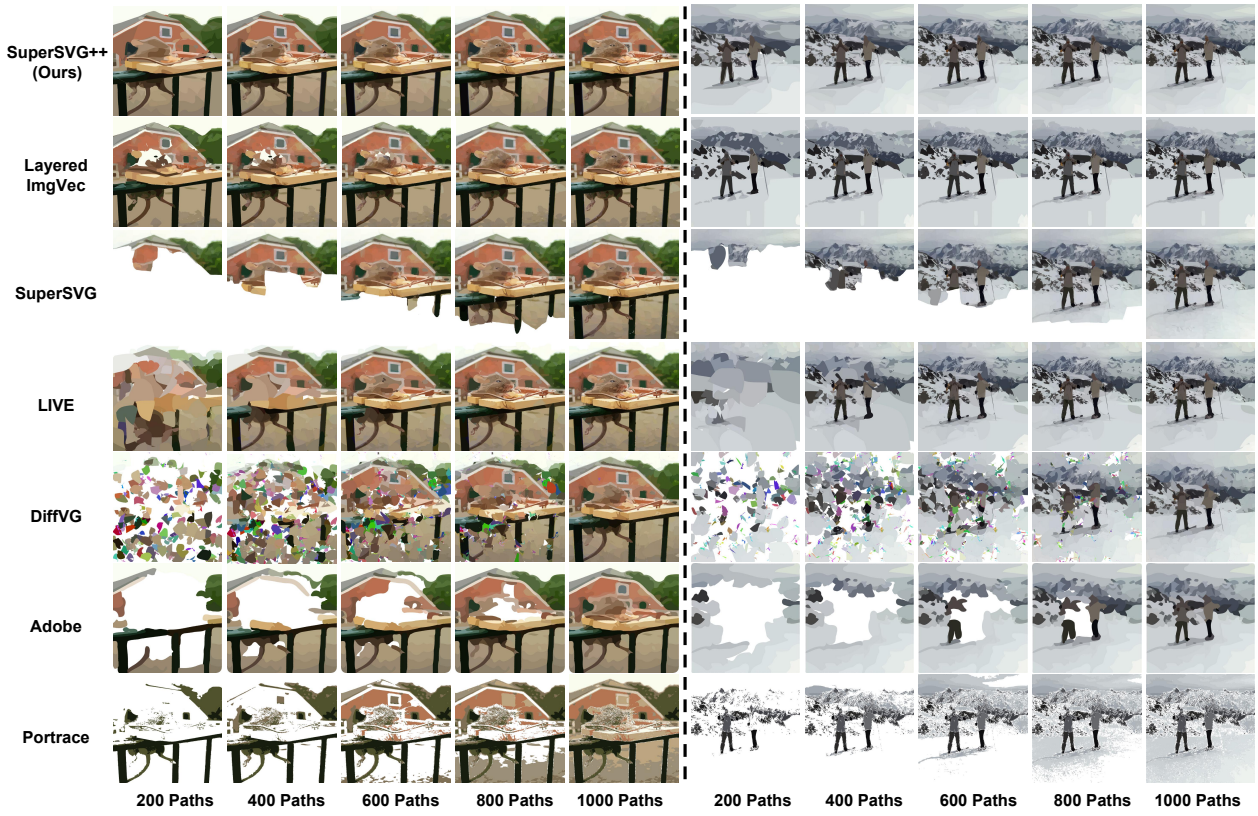


Fig. 9: Qualitative comparison on the layer-wise effect between our SuperSVG++ and the state-of-the-art methods. It can be seen that SuperSVG++ can vectorize the image in a coarse-to-fine manner with a good layer-wise effect, which is closer to the man-made SVGs and is good for downstream tasks like SVG editing.

niSVG [33], and StarVector [32]. As shown in Fig. 10, our SuperSVG++ achieves robust vectorization performance across different types of graphic images. For simple icons, all methods can roughly capture the main structure, while our method produces cleaner shapes and more faithful details. For more complex SVG-style characters, flowcharts, and posters, Layered ImgVec can preserve the overall layout but tends to lose fine local structures and detailed visual elements. In contrast, our SuperSVG++ reconstructs more complete object contours, richer local details, and more faithful color regions. OmniSVG and StarVector can generate coarse SVG results for simple icons, but their performance degrades significantly on complex graphic designs, sometimes recovering only a small portion of the target image or even producing nearly blank results. These results demonstrate that SuperSVG++ is not limited to ImageNet-style natural images, but can also generalize well to practical SVG vectorization scenarios such as icons, characters, flowcharts, and posters. Meanwhile, we observe that small text regions in flowcharts and posters remain challenging for all compared methods, including ours. Accurately preserving tiny characters and dense text structures in SVG vectorization remains an open problem.

5.2 Layer-wise Image Vectorization Comparison

Evaluation metric on layer-wise effect. SuperSVG++ aims to vectorize images x in a layer-wise manner. However, there is no metric to evaluate the layer-wise effect of the vectorization results. In SVG, different paths contribute to image details and semantic components in a hierarchical way. By evaluating how the cumulative addition of paths at different "layers" affects the similarity to the target image, we can assess the layer-wise effect. Therefore, we propose a new metric named Layer-wise Score (LS). Specifically, for an SVG with n paths $\{s_i\}_{i=1}^{n}$, we split the n paths into k layers $\{S_1, S_2, \cdots S_k\}$, where layer j contains the first $\lfloor \frac{n}{k} j \rfloor$ paths. Then, we render the paths in each layer $\{S_1, S_2, \cdots S_k\}$ into an image for each layer $\{y_1, y_2, \cdots y_k\}$. Finally, we compute LS by:

$$LS = \frac{1}{k} \sum_{i=1}^{k} \|E(y_i) - E(x)\|. \tag{6}$$

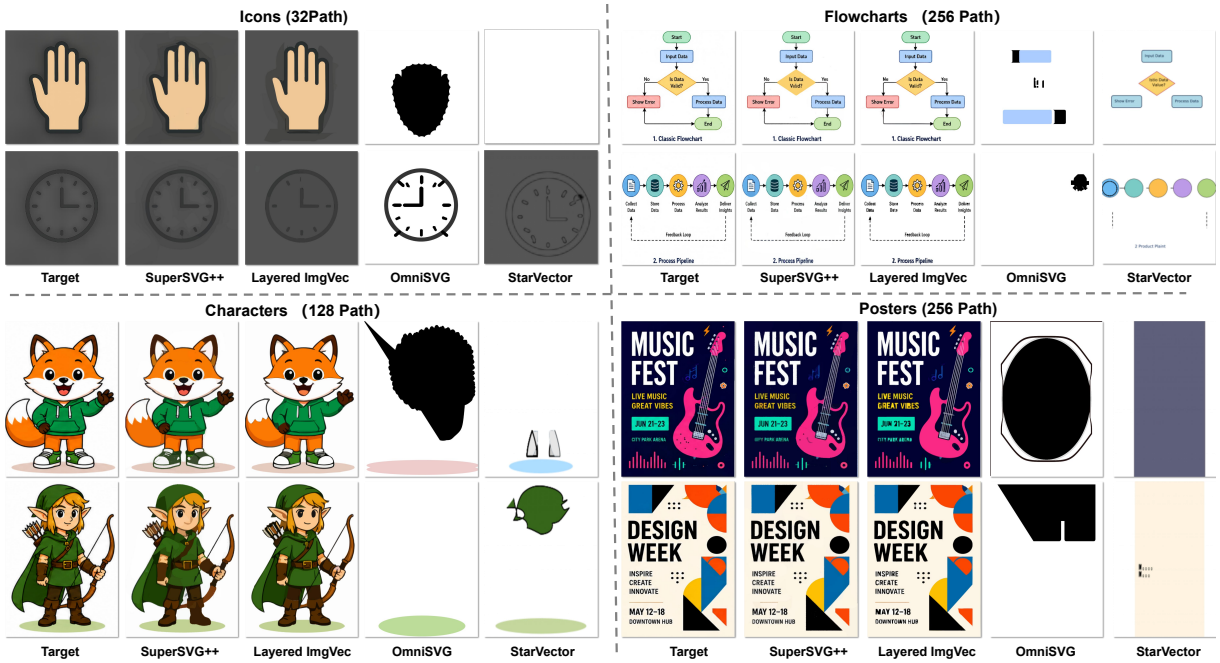


Fig. 10: Qualitative comparison on SVG-style graphic images, including simple icons, flowcharts, cartoon characters, and posters. The results demonstrate the superior generalization ability of our model across practical SVG-style vectorization scenarios.

where $E(\cdot)$ is the image encoder [50] from LPIPS [51]. The lower the LS value, the better the layer-wise effect.

Comparison between existing methods on layer-wise effect. We compare our SuperSVG++ with the state-of-the-art image vectorization methods on the layer-wise effect, including SuperSVG [17], LIVE [14], SGLIVE [18], Layered ImgVec [20], Optimize&Reduce [19], DiffVG [13], Adobe [27], and Potrace [1]. Specifically, we choose 1000-paths results with $k = 5$ layers for comparison. The qualitative comparison results are shown in Fig. 9. It can be seen that SuperSVG vectorizes the target image superpixel by superpixel, which has a bad layer-wise effect. Both Adobe and Potrace have similar results, limiting their performance in layer-wise vectorization. Moreover, since DiffVG randomly samples the initial path parameters and optimizes them all together, the lower layers of DiffVG have many artifacts caused by the paths covered by the top layer. LIVE has a relatively better layer-wise effect, but it can be seen that, without semantic awareness, the first two layers of LIVE only consider the pixel-level distance, causing substantial visual differences from the target image. Layered ImgVec shows a better layer-wise effect than the above methods by explicitly modeling layered vectorization. However, as shown in the left example of Fig. 9, it may still miss part of the semantic structure in lower layers when handling images with complex details. For example, the mouse head is not fully rendered before 800 paths, indicating that some important object-level semantics are delayed or missing in the early layers. In comparison, our SuperSVG++ can construct the main semantic information of the target image in this first layer with the help of our semantic-aware coarse reconstruction method. In addition, our method gradually refines the details by our layer-wise superpixel decomposition strategy, which contributes to both the best layer-wise effect and vectorization accuracy.

Moreover, to demonstrate the effectiveness of our SuperSVG++ in layer-wise vectorization, we compute the layer-wise score for all these compared methods under 500, 1000, and 2000 paths with $m = 5$ layers. The quantitative results are shown in Tab. 4. It shows that our SuperSVG++ achieves the best layer-wise score, indicating the best layer-wise effect across all baselines.

Vector Compactness. Furthermore, to quantify the layer-wise representation and assess how well the generated paths are contained within the semantic object boundaries, we report the Vector Compactness (VeC) metric [20]. Given a semantic mask of a target object, VeC is defined as the ratio of paths highly contained within the mask (i.e., exceeding 85% area overlap) to the total number of paths interacting with the mask. A higher VeC score indicates a more compact and semantically accurate vector representation.

We compute the VeC scores for our method and the representative baselines under 500, 1000, and 2000 paths.

Table 2: Quantitative comparison between the state-of-the-arts and Ours on ImageNet. **Bold** and underline for the best and the second best results.

#Paths	Method	Time (s) ↓	MSE ↓	PSNR ↑	LPIPS ↓	SSIM ↑
-	StarVector [32]	89.29	0.3254	5.50	0.6409	0.3905
-	OmniSVG-8B [33]	66.77	0.2566	6.77	0.6736	0.2302
500	LIVE [14]	18,971	0.0039	24.10	0.4467	0.7983
	SGLIVE [18]	1,244	0.0532	14.76	0.4723	0.6926
	DiffVG [13]	19.29	0.0069	21.42	0.5319	0.6671
	Adobe [27]	0.87	0.0067	21.82	0.5595	0.6939
	Potrace [1]	0.98	0.0208	17.85	0.5115	0.6920
	VTracer [49]	1.01	0.0108	20.10	0.4853	0.7230
	Optimize&Reduce [19]	90.10	0.0102	20.91	0.4860	0.5370
	Layered ImgVec [20]	237.10	0.0036	26.21	0.4178	0.8146
	SuperSVG [17]	**0.31**	0.0044	24.80	0.4452	0.7687
	SuperSVG++ (Ours)	2.53	0.0041	25.32	0.4428	0.7881
	SuperSVG++* (Ours)	10.00	**0.0025**	**27.58**	**0.4090**	**0.8336**
1,000	LIVE [14]	31,052	0.0030	26.61	0.4341	0.8230
	SGLIVE [18]	2,720	0.0416	15.31	0.4419	0.7312
	DiffVG [13]	35.05	0.0039	25.04	0.4812	0.7751
	Adobe [27]	1.18	0.0057	23.48	0.4696	0.7466
	Potrace [1]	1.26	0.0167	19.57	0.4409	0.6807
	VTracer [49]	4.18	0.0039	25.15	0.3779	0.8359
	Optimize&Reduce [19]	181.30	0.0102	20.89	0.4862	0.6770
	Layered ImgVec [20]	563.21	0.0024	27.75	0.3638	0.8589
	SuperSVG [17]	**0.52**	0.0032	26.03	0.4075	0.8111
	SuperSVG++ (Ours)	3.52	0.0027	26.91	0.3729	0.8307
	SuperSVG++* (Ours)	10.00	**0.0019**	**28.91**	**0.3495**	**0.8664**
2,000	LIVE [14]	117,928	0.0025	26.98	0.3994	0.8431
	SGLIVE [18]	7,860	0.0341	16.03	0.4162	0.7441
	DiffVG [13]	73.65	0.0036	25.88	0.4683	0.7710
	Adobe [27]	2.15	0.0033	26.23	0.3961	0.7229
	Potrace [1]	3.10	0.0160	19.65	0.4355	0.6997
	VTracer [49]	12.71	0.0023	27.53	0.3103	0.8559
	Optimize&Reduce [19]	273.74	0.0101	20.93	0.4852	0.6781
	Layered ImgVec [20]	1,031	0.0016	29.16	0.3290	0.8797
	SuperSVG [17]	**0.71**	0.0024	27.25	0.3648	0.8446
	SuperSVG++ (Ours)	5.10	0.0020	28.37	0.3231	0.8688
	SuperSVG++* (Ours)	10.00	**0.0015**	**29.88**	**0.2981**	**0.8935**

Table 3: Quantitative comparison on MMSVGBench image2svg split (300 images). **Bold** and underline for the best and the second best results.

#Paths	Method	Time (s) ↓	MSE ↓	PSNR ↑	LPIPS ↓	SSIM ↑
-	StarVector [32]	89.29	0.1800	8.20	0.6200	0.4200
-	OmniSVG-8B [33]	66.77	0.1450	9.10	0.6500	0.3100
32	LIVE [14]	1,213	0.0049	25.10	0.3370	0.8720
	SGLIVE [18]	79.60	0.0412	16.35	0.3910	0.7790
	DiffVG [13]	5.20	0.0061	23.49	0.3246	0.8602
	Adobe [27]	0.76	0.0120	21.90	0.3301	0.8810
	Potrace [1]	0.24	0.0555	14.35	0.4444	0.7760
	VTracer [49]	0.19	0.0111	22.69	0.3598	0.8904
	Optimize&Reduce [19]	27.13	0.0052	24.70	0.3326	0.8694
	Layered ImgVec [20]	15.20	0.0024	28.40	0.3000	0.9050
	SuperSVG [17]	**0.12**	0.0018	30.10	0.3050	0.9010
	SuperSVG++ (Ours)	1.53	0.0015	31.03	0.2929	0.9085
	SuperSVG++* (Ours)	10.00	**0.0014**	**31.20**	**0.2910**	**0.9100**
64	LIVE [14]	2,829	0.0015	30.48	0.3040	0.9170
	SGLIVE [18]	162.20	0.0380	16.80	0.3860	0.7860
	DiffVG [13]	8.80	0.0035	25.57	0.3444	0.8632
	Adobe [27]	0.77	0.0101	22.10	0.3518	0.8689
	Potrace [1]	0.37	0.0310	15.75	0.4378	0.7810
	VTracer [49]	0.36	0.0095	22.97	0.3536	0.8941
	Optimize&Reduce [19]	46.19	0.0050	24.79	0.3320	0.8698
	Layered ImgVec [20]	30.30	0.0012	31.10	0.2960	0.9188
	SuperSVG [17]	**0.15**	0.0008	32.80	0.3010	0.9180
	SuperSVG++ (Ours)	1.72	0.0006	33.72	0.2909	0.9249
	SuperSVG++* (Ours)	10.00	**0.0006**	**33.90**	**0.2890**	**0.9260**
128	LIVE [14]	5,454	0.0008	32.70	0.3010	0.9240
	SGLIVE [18]	350.50	0.0345	17.30	0.3800	0.7950
	DiffVG [13]	14.80	0.0016	28.60	0.3658	0.8739
	Adobe [27]	0.82	0.0091	23.15	0.3420	0.8890
	Potrace [1]	0.45	0.0213	17.45	0.4310	0.7952
	VTracer [49]	0.47	0.0083	23.20	0.3463	0.8961
	Optimize&Reduce [19]	57.10	0.0049	24.78	0.3323	0.8700
	Layered ImgVec [20]	60.70	0.0005	33.90	0.2874	0.9372
	SuperSVG [17]	**0.23**	0.0004	35.10	0.2980	0.9320
	SuperSVG++ (Ours)	1.90	0.0003	35.86	0.2877	0.9366
	SuperSVG++* (Ours)	10.00	**0.0003**	**36.10**	**0.2860**	**0.9380**

The quantitative results are summarized in Tab. 5. As shown in the table, methods based on random path initialization and global optimization, such as DiffVG, tend to generate less compact paths that extend across object boundaries and cover pixels from different semantic regions. This makes each path less well aligned with a coherent object region, resulting in lower compactness scores. While Layered ImgVec achieves competitive results by explicitly optimizing for layered structures, our SuperSVG++ consistently achieves the best VeC scores across all path settings. This demonstrates that our semantic-aware coarse-to-fine framework not only reconstructs better visual details and layer-wise effects but also generates highly compact and efficient SVG representations.

5.3 SVG Editing

Comparison on object-level SVG editing. Our SuperSVG++ not only excels at image vectorization, but also facilitates downstream SVG editing through the proposed semantic-aware reconstruction process and soft-hard boundary constraint. In Fig. 11, we compare SVG editing results using the vectorized outputs from LIVE, DiffVG, and our method by moving the vectorized truck to its left. Specifically, we provide a semantic mask for the editing target and identify the paths pri-

Table 4: Quantitative comparison on the layer-wise effect evaluated by the proposed layer-wise score LS (↓) between the state-of-the-art methods and Our SuperSVG++.

Method	# Path Number		
	500	1,000	2,000
LIVE [14]	0.4791	0.4519	0.4163
SGLIVE [18]	0.5518	0.5432	0.5340
DiffVG [13]	0.5847	0.5938	0.6103
Adobe Illustrator [27]	0.5511	0.5424	0.5232
Potrace [1]	0.5529	0.5054	0.5062
Optimize&Reduce [20]	0.5318	0.5312	0.5310
Layered ImgVec [20]	0.4727	0.4395	0.4114
SuperSVG [17]	0.5554	0.5370	0.5234
SuperSVG++ (Ours)	**0.4696**	**0.4380**	**0.4063**

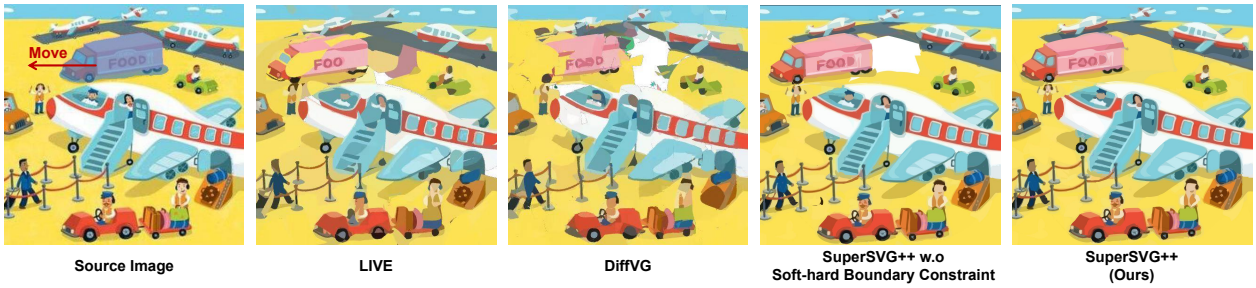


Fig. 11: Comparison on SVG editing by moving the vectorized truck from LIVE, DiffVG, and our SuperSVG++. The results show that the generated SVGs from both LIVE and DiffVG are not well suited to SVG editing, leading to artifacts after editing. In comparison, with our semantic-aware vectorization process, our SuperSVG++ can identify the semantics well, which supports accurate editing for the target semantics. Moreover, with the help of our soft-hard boundary constraint, the area of the edited semantics remains the color of the background, making the edited effect more natural.

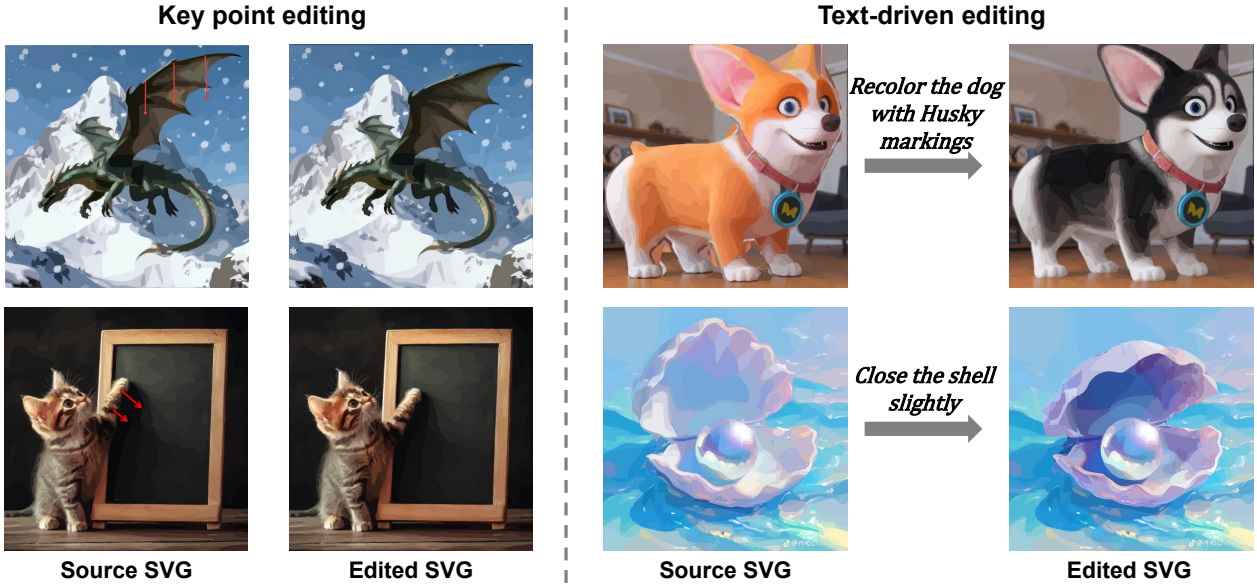


Fig. 12: More SVG editing results of SuperSVG++. The left part shows TPS-based non-rigid deformation results, where users can select the target semantic layer and deform its SVG paths while keeping the remaining layers unchanged. The right part shows text-driven editing results, where an image-editing diffusion model is used as guidance to optimize the SVG paths of the selected layer. These results demonstrate that our layer-wise SVG representation supports both geometry-aware deformation and semantic text-driven editing in a controllable manner.

marily located within the mask, *i.e.*, paths covering more than 80% of the mask area. These selected paths are then modified according to the specified editing operation, such as translation or rotation. As shown in Fig. 11, SVGs generated by LIVE and DiffVG are not well suited to seamless editing, resulting in noticeable artifacts after moving the target object. In contrast, our semantic-aware vectorization process enables SuperSVG++ to better capture object-level semantics, thereby supporting more accurate editing of the target object. Moreover, with the help of the soft-hard boundary constraint, the background region originally occluded by the edited object is filled with consistent background colors, producing a more natural editing effect.

More editing results. In addition to the above object-level translation experiment, we further show more flexible SVG editing results in Fig. 12. For complex editing scenarios, our framework can be extended with semantic layering models, such as Qwen-Image-Layered [47], to obtain semantically disentangled layers and their corresponding alpha masks. Given these layer masks, users can conveniently select the target layer to perform layer-wise SVG editing while keeping the other layers

Table 5: Quantitative comparison on the Vector Compactness (VeC) metric. Higher values indicate better compactness. **Bold** and underline denote the best and the second-best results, respectively.

Method	Vector Compactness (VeC) ↑		
	500 Paths	1000 Paths	2000 Paths
Potrace [1]	0.495	0.521	0.552
Adobe Illustrator [27]	0.512	0.533	0.550
DiffVG [13]	0.479	0.541	0.560
LIVE [14]	0.490	0.555	0.581
Optimize&Reduce [19]	0.508	0.536	0.562
SGLIVE [18]	0.608	0.634	0.681
Layered ImgVec [20]	0.601	0.644	0.661
SuperSVG [17]	0.530	0.554	0.585
SuperSVG++ (Ours)	**0.685**	**0.705**	**0.724**

unchanged. For geometry-aware editing, we apply thin-plate spline (TPS) transformation to the paths belonging to the selected layer, which enables non-rigid deformation of the target object or region while preserving the remaining SVG layers. For instruction-driven editing, users can first select the target semantic layer and then employ a text-driven image editing model, such as GPT-Image, to generate the edited raster target according to the text instruction. After that, we optimize only the SVG paths of the selected layer toward the edited target using the multi-scale distance loss and boundary constraint, while keeping the other layers fixed. In this way, the target semantic layer can be edited according to the user instruction, and the original scene structure outside the edited layer can be well preserved. As shown in Fig. 12, our layer-wise SVG representation supports diverse editing operations, including object translation, non-rigid deformation, and instruction-driven semantic editing, demonstrating the practical editability of SuperSVG++.

5.4 Runtime Analysis

Evaluation setup for runtime analysis. To evaluate the computational efficiency of our method, we conduct a comprehensive runtime analysis across different image resolutions (256×256, 512×512, and 1024×1024) and target path counts (500, 1000, and 2000). The runtime is measured by generating a single vector image, averaged over 100 test images.

Comparison between existing methods on runtime. We compare our SuperSVG++ with traditional algorithmic methods, optimization-based methods, and deep-learning-based methods. As shown in Tab. 6, the runtime of most methods increases as the image resolution and target path count become larger. This trend is particularly significant for optimization-based methods, such as DiffVG [13], LIVE [14], SGLIVE [18], Optimize&Reduce [19], and Layered ImgVec [20], since they require iterative optimization over more path parameters and repeated differentiable rendering at higher resolutions. As a result, these methods can take from tens of seconds to minutes or even hours to vectorize a single high-resolution image, especially under large-path settings. Although traditional algorithmic methods such as Potrace [1] and Adobe Illustrator [27] are computationally efficient, they lack explicit semantic modeling and struggle with complex color transitions and object-level structures, which limits their performance in semantic-aware layer-wise vectorization.

Among deep-learning-based methods, StarVec [32] and OmniSVG [33] rely on large generative models and therefore introduce noticeable inference latency. SuperSVG [17] achieves the fastest inference due to its lightweight coarse-to-fine prediction pipeline, but it does not explicitly preserve layer-wise semantic structures. In comparison, our SuperSVG++ introduces semantic-aware coarse reconstruction and layer-wise refinement modules, requiring slightly more runtime than SuperSVG. Nevertheless, SuperSVG++ still maintains second-level runtime across all tested resolutions and path budgets. For example, it takes only 3.52 seconds for 1000 paths at 512×512 resolution and 6.52 seconds even for 2000 paths at 1024×1024 resolution. Compared with most optimization-based methods, SuperSVG++ is one to several orders of magnitude faster while delivering state-of-the-art layer-wise vectorization quality and editing capability. These results demonstrate that SuperSVG++ achieves a favorable balance between computational efficiency, vectorization accuracy, and semantic layer-wise editability.

5.5 Comparison with Qwen-Image-Layered Integration

To further evaluate the effect of external semantic layering models, we compare the default SuperSVG++ pipeline with the Qwen-Image-Layered integrated variant described in Sec. 4.3. The remaining vectorization stages are kept unchanged to ensure a fair comparison. Besides standard reconstruction metrics, we further conduct a user study to evaluate editing quality. Specifically, we use the SVGs produced by both variants for the same editing operations and invite 18 users to rank the edited results according to their visual quality and editing naturalness. The normalized preference score is reported as the Editing score in Table 7.

As shown in Table 7, integrating Qwen-Image-Layered significantly increases the inference time, since semantic layer decomposition is itself a generative process performed before vectorization. Meanwhile, the

Table 6: Comparison of runtime (in seconds) across different image resolutions and target path counts. The runtime is averaged over 100 images. "N/A" indicates the method does not utilize a learnable neural network.

Method	Params	Resolution 256			Resolution 512			Resolution 1024		
		500 Paths	1k Paths	2k Paths	500 Paths	1k Paths	2k Paths	500 Paths	1k Paths	2k Paths
Potrace [1]	N/A	0.50	0.64	1.48	0.98	1.26	3.10	2.54	3.28	7.82
Adobe Illustrator [27]	N/A	0.45	0.62	1.07	0.87	1.18	2.15	2.18	3.04	5.62
StarVec [32]	7.73B		76.40			89.29			112.10	
OmniSVG [33]	8B		46.30			66.77			90.75	
DiffVG [13]	N/A	9.30	18.10	37.40	19.29	35.05	73.65	47.80	91.30	188.60
LIVE [14]	N/A	9,240	16,080	60,150	18,971	31,052	117,928	48,600	79,850	291,300
Optimize&Reduce [19]	N/A	43.60	92.10	137.90	90.10	181.30	273.74	229.80	447.60	704.20
Layered ImgVec [20]	N/A	121.30	272.50	536.80	237.10	563.21	1,031	604.20	1,486.50	2,642.70
SGLIVE [18]	N/A	605	1,335	3,820	1,244	2,720	7,860	3,080	6,940	19,120
SuperSVG [17]	53.90M	0.26	0.43	0.61	0.31	0.52	0.71	0.38	0.66	0.92
SuperSVG++ (Ours)	102.67M	2.12	3.05	4.28	2.53	3.52	5.10	3.18	4.34	6.52

Table 7: Comparison between the default SuperSVG++ pipeline and the variant integrated with Qwen-Image-Layered. Qwen-Image-Layered introduces higher inference cost and slightly lower reconstruction accuracy, but achieves better editing quality due to occlusion completion.

Paths	Method	Time (s) ↓	MSE ↓	PSNR ↑	LPIPS ↓	SSIM ↑	Editing ↑
500	SuperSVG++	**2.53**	**0.0041**	**25.32**	**0.4428**	**0.7881**	0.1370
	SuperSVG++ w/ Qwen-IL	511.57	0.0045	24.91	0.4556	0.7768	**0.8630**
1,000	SuperSVG++	**3.52**	**0.0027**	**26.91**	**0.3729**	**0.8307**	0.1074
	SuperSVG++ w/ Qwen-IL	504.22	0.0030	26.48	0.3854	0.8192	**0.8926**
2,000	SuperSVG++	**5.10**	**0.0020**	**28.37**	**0.3231**	**0.8688**	0.0851
	SuperSVG++ w/ Qwen-IL	507.10	0.0023	27.92	0.3358	0.8569	**0.9149**

reconstruction quality is slightly worse than the default SuperSVG++ pipeline under the same path budget. This is because Qwen-Image-Layered completes invisible or occluded regions for each semantic layer, and these additional inpainted contents are also vectorized as valid layer-wise targets. Consequently, part of the path budget is spent on hidden regions rather than only reconstructing the visible image content, leading to slightly degraded MSE, PSNR, LPIPS, and SSIM.

However, this occlusion completion ability is beneficial for downstream SVG editing. When foreground objects are moved or removed, previously occluded regions may become exposed. The default SuperSVG++ pipeline mainly reconstructs visible content, while the Qwen-Image-Layered variant provides more complete underlying layers through inpainting. To verify this advantage, we further conduct a user study on editing quality. Specifically, we select 15 images and perform the same object-level editing operations on the SVGs generated by the default SuperSVG++ pipeline and the Qwen-Image-Layered variant. We then invite 18 users to compare the edited results and indicate which one has better visual quality and editing naturalness. The normalized preference score is reported as the Editing score in Table 7. As shown in the table, although the Qwen-Image-Layered variant slightly sacrifices reconstruction fidelity, it achieves much higher Editing scores, demonstrating its advantage in producing more natural edited SVG results. Overall, the default SuperSVG++ pipeline provides a better balance for efficient and accurate image-to-SVG reconstruction, while the Qwen-Image-Layered integration can serve as an optional editability-oriented variant.

5.6 Ablation Study

Ablation on the soft-hard boundary constraint. We first perform an ablation study on our soft-hard boundary constraint in the coarse stage of SuperSVG++, which allows the coarse path to extend beyond the soft-constraint boundary. This ensures that the predicted paths are more compact and better aligned with the target semantic area. With this approach, background paths can extend through the foreground boundaries, ensuring that when the foreground object is moved, the original location of the object remains consistent with the background color. This results in a more natural SVG editing effect. To validate this, we conducted an SVG editing experiment in Fig. 11 by shifting the truck to its left. Without our soft-hard boundary constraint, moving the truck leaves large blank areas in the background, leading to poor editing results. In contrast, our approach achieves accurate and natural edits, highlighting the importance of the soft-hard boundary constraint.

Ablation on the proposed modules. We conduct more ablation studies on the proposed difference-conditioned module (DCM), multi-level distance loss ($\mathcal{L}_{MD}$), and the layer-wise decomposition strategy (LDS) in SuperSVG++. We train four ablated models: (1) model without DCM; (2) model without $\mathcal{L}_{MD}$ and $\mathcal{L}_{DPW}$; (3) the model without $\mathcal{L}_{MD}$ only; and (4) model without LDS. Then, we compare these ablated models with our SuperSVG++ under 1,000 paths, and the re-

Table 8: Ablation study on the proposed difference-conditioned module (DCM), multi-level distance loss ($\mathcal{L}_{MD}$), and the layer-wise decomposition strategy (LDS)

Method	MSE ↓	PSNR ↑	LPIPS ↓	SSIM ↑
SuperSVG++ (Ours)	**0.0027**	**26.91**	**0.3729**	**0.8307**
w.o. DCM	0.0044	25.03	0.4238	0.7612
w.o. $\mathcal{L}_{MD}$,$\mathcal{L}_{DPW}$	0.0201	17.44	0.5215	0.6841
w.o. $\mathcal{L}_{MD}$ only	0.0035	25.71	0.4062	0.8010
w.o. LDS	0.0030	26.43	0.3810	0.8105

sults are shown in Tab. 8. It can be seen that the model without the difference-conditioned module gets a much worse vectorization accuracy than SuperSVG++. Moreover, the model without $\mathcal{L}_{MD}$ and $\mathcal{L}_{DPW}$ cannot be well trained, thereby resulting in the worst vectorization results. The model without $\mathcal{L}_{MD}$ only (with $\mathcal{L}_{DPW}$) can be trained to refine the details, but have a worse result than SuperSVG++ due to the local view field of the ablated model. Finally, the model without LDS achieves slightly worse reconstruction results than SuperSVG++, but gets a much worse layer-wise effect (Fig. 13).

Ablation on hyperparameter sensitivity. To further examine whether SuperSVG++ is sensitive to manually designed hyperparameters, we conduct a sensitivity analysis on five key hyperparameters, including the boundary loss weight λ_B, the SAM mask granularity, the downsample scale factor k in $\mathcal{L}_{MD}$, the number of selected masks per refinement layer m, and the number of decomposed superpixels n for each selected mask. For each hyperparameter, we vary it around the default setting while keeping all the other settings unchanged. Specifically, the default configuration uses the intermediate setting among the tested values, i.e., $\lambda_B = 1.0\times$, medium SAM mask granularity, $k = 4$, $m = 10$, and $n = 6$. As shown in Tab. 9, changing these hyperparameters within a reasonable range only leads to minor performance fluctuations. For example, when varying λ_B from $0.5\times$ to $2.0\times$, the MSE remains around 0.0028-0.0029, and the PSNR remains close to the default result. Similarly, different SAM mask granularities, downsample factors, selected mask numbers, and superpixel numbers all achieve comparable performance. These results indicate that SuperSVG++ is not sensitive to specific hyperparameter choices. Instead, the performance gain mainly comes from the proposed soft-hard boundary constraint, difference-conditioned refinement, multi-scale distance supervision, and layer-wise refinement strategy, rather than carefully tuned hyperparameters.

Table 9: Sensitivity analysis of key hyperparameters in SuperSVG++. We vary one hyperparameter at a time while keeping the others fixed to the default setting: $\lambda_B = 1.0\times$, medium SAM mask granularity, downsample factor $k = 4$ in $\mathcal{L}_{MD}$, selected masks per layer $m = 10$, and superpixels per mask $n = 6$.

Hyperparameter	Setting	MSE ↓	PSNR ↑	LPIPS ↓	SSIM ↑
Default	SuperSVG++ (Ours)	**0.0027**	**26.91**	**0.3729**	**0.8307**
Boundary loss weight λ_B	$0.5\times$	0.0028	26.82	0.3741	0.8288
	$2.0\times$	0.0029	26.79	0.3752	0.8279
SAM mask granularity	Coarse	0.0029	26.65	0.3796	0.8248
	Fine	0.0028	26.76	0.3762	0.8271
Scale factor k in $\mathcal{L}_{MD}$	$k = 2$	0.0028	26.86	0.3744	0.8292
	$k = 8$	0.0029	26.70	0.3780	0.8262
Selected masks per layer m	$m = 5$	0.0029	26.68	0.3787	0.8256
	$m = 20$	0.0028	26.84	0.3749	0.8291
Superpixels per mask n	$n = 4$	0.0029	26.72	0.3776	0.8267
	$n = 8$	0.0028	26.81	0.3755	0.8284

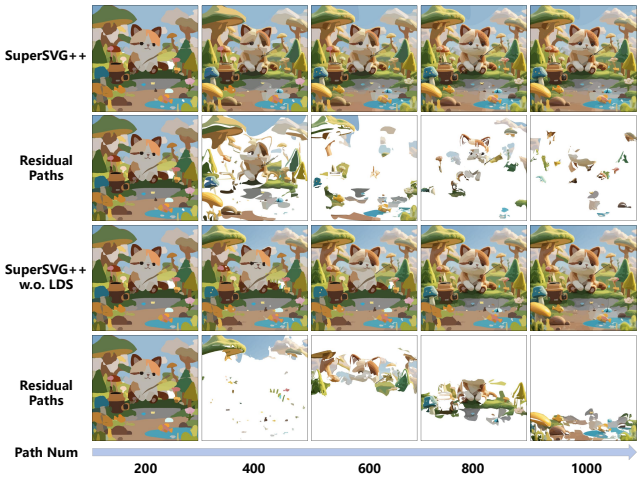


Fig. 13: Ablation on the layer-wise decomposition strategy (LDS). For SuperSVG++ and the ablated version without LDS, the first row shows the layer-wise SVGs with different path numbers, and the second row shows the residual paths between different layers. It can be seen that the model without LDS generates paths from the left-top to the right-bottom, while our SuperSVG++ with LDS generates paths from large to small, contributing to a layer-wise effect.

6 Conclusions and Limitations

Conclusions. In this paper, we proposed SuperSVG++, a semantic-driven, layer-wise vectorization model that enhances both the accuracy and efficiency of scalable vector graphics synthesis. By leveraging Segment Anything and Depth Anything models, our method extracts semantic layers in a depth-aware manner, ensuring structured and meaningful vectorization. Additionally, we introduced a soft-hard boundary constraint for compact path generation, a difference-conditioned refinement

model for precise detail enhancement, and a layer-wise refinement strategy for hierarchical refinement. Extensive experiments demonstrate that SuperSVG++ outperforms state-of-the-art methods in both reconstruction accuracy and layer-wise coherence, achieving a more structured and editable vector representation. Our approach provides a robust foundation for vector-based image generation and editing, with potential applications in graphic design and digital content creation.

Limitations. The editing capability of the default SuperSVG++ pipeline is constrained when an object is heavily or structurally occluded. The default pipeline reconstructs SVG layers primarily from visible image evidence and does not explicitly infer the complete geometry and appearance of hidden object parts. Although the soft-hard boundary constraint improves background continuity when a foreground object is moved, translating, rotating, or deforming a partially occluded object may expose incomplete or inaccurate shapes under complex occlusion. The optional Qwen-Image-Layered integration can alleviate this issue by generating occlusion-completed layer targets, but it introduces substantial computational cost and its editing quality depends on the reliability of the generated hidden content. Developing efficient and robust native shape completion for occluded objects remains an important direction for future work.

7 Data Availability Statement

The datasets used in this study are publicly available. We utilized the **ImageNet** dataset for both training and evaluation. The training process employed the official ImageNet training split, while the quantitative results were evaluated on a fixed subset of 1,000 images from the ImageNet validation set. These resources are available for download at http://www.image-net.org.

Funding This work was supported by National Key R&D Program of China (2024YFE0115500), and National Natural Science Foundation of China (No. 62302297, 625B2115).

References

1. Peter Selinger. Potrace: a polygon-based tracing algorithm, 2003.
2. Hailing Zhou, Jianmin Zheng, and Lei Wei. Representing images using curvilinear feature driven subdivision surfaces. *IEEE Transactions on Image Processing*, 23(8):3268–3280, Aug 2014.
3. Zicheng Liao, H. Hoppe, D. Forsyth, and Yizhou Yu. A subdivision-based representation for vector image editing. *IEEE Transactions on Visualization and Computer Graphics*, 18(11):1858–1867, Nov 2012.
4. Yu-Kun Lai, Shi-Min Hu, and Ralph R. Martin. Automatic and topology-preserving gradient mesh generation for image vectorization. *ACM Transactions on Graphics*, 28(3):1–8, Jul 2009.
5. Sebastian Thrun and James Diebel. Bayesian image vectorization: the probabilistic inversion of vector image rasterization. Jan 2008.
6. Ming Yang, Hongyang Chao, Chi Zhang, Jun Guo, Lu Yuan, and Jian Sun. Effective clipart image vectorization through direct optimization of bezigons. *IEEE Transactions on Visualization and Computer Graphics*, page 1063–1075, Feb 2016.
7. Pradyumna Reddy, Michael Gharbi, Michal Lukac, and Niloy J Mitra. Im2Vec: Synthesizing vector graphics without vector supervision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7342–7351, 2021.
8. Chen Liu, Jiajun Wu, Pushmeet Kohli, and Yasutaka Furukawa. Raster-to-vector: Revisiting floorplan transformation. In *2017 IEEE International Conference on Computer Vision (ICCV)*, Oct 2017.
9. Jun Gao, Chengcheng Tang, Vignesh Ganapathi-Subramanian, Jiahui Huang, Hao Su, and LeonidasJ. Guibas. DeepSpline: Data-driven reconstruction of parametric curves and surfaces. Jan 2019.
10. David Ha and Douglas Eck. A neural representation of sketch drawings. *Learning,Learning*, Apr 2017.
11. Leo Sampaio Ferraz Ribeiro, Tu Bui, John Collomosse, and Moacir Ponti. Sketchformer: Transformer-based representation for sketched structure. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun 2020.
12. Teng Hu, Ran Yi, Haokun Zhu, Liang Liu, Jinlong Peng, Yabiao Wang, Chengjie Wang, and Lizhuang Ma. Stroke-based neural painting and stylization with dynamically predicted painting region. In *Proceedings of the 31st ACM International Conference on Multimedia*, pages 7470–7480, 2023.
13. Tzu-Mao Li, Michal Lukáč, Michaël Gharbi, and Jonathan Ragan-Kelley. Differentiable vector graphics rasterization for editing and learning. *ACM Transactions on Graphics (TOG)*, 39(6):1–15, 2020.
14. Xu Ma, Yuqian Zhou, Xingqian Xu, Bin Sun, Valerii Filev, Nikita Orlov, Yun Fu, and Humphrey Shi. Towards layer-wise image vectorization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16314–16323, 2022.
15. Haokun Zhu, Juang Ian Chong, Teng Hu, Ran Yi, Yu-Kun Lai, and Paul L Rosin. SAMVG: A multi-stage image vectorization model with the segment-anything model. *arXiv preprint arXiv:2311.05276*, 2023.
16. Bohao Tang, Teng Hu, Yuzhen Du, Ran Yu, and Lizhuang Ma. Curved-stroke-based neural painting and stylization through thin plate spline interpolation. *Scientia Sinica Informationis*, 54(2):301–315, 2024.
17. Teng Hu, Ran Yi, Baihong Qian, Jiangning Zhang, Paul L Rosin, and Yu-Kun Lai. SuperSVG: Superpixel-based scalable vector graphics synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 24892–24901, 2024.
18. Hengyu Zhou, Hui Zhang, and Bin Wang. Segmentation-guided layer-wise image vectorization with gradient fills. In *European Conference on Computer Vision*, pages 165–180. Springer, 2024.
19. Or Hirschorn, Amir Jevnisek, and Shai Avidan. Optimize & reduce: a top-down approach for image vectorization.

In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 2148–2156, 2024.
20. Zhenyu Wang, Jianxi Huang, Zhida Sun, Yuanhao Gong, Daniel Cohen-Or, and Min Lu. Layered image vectorization via semantic simplification. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 7728–7738, 2025.
21. Kaibo Zhao, Liang Bao, Yufei Li, Xu Su, Ke Zhang, and Xiaotian Qiao. Less is more: Efficient image vectorization with adaptive parameterization. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 18166–18175, 2025.
22. Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4015–4026, 2023.
23. Lihe Yang, Bingyi Kang, Zilong Huang, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything: Unleashing the power of large-scale unlabeled data. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10371–10381, 2024.
24. Shuang Zhao, Frédo Durand, and Changxi Zheng. Inverse diffusion curves using shape optimization. *IEEE Transactions on Visualization and Computer Graphics*, Oct 2016.
25. Guofu Xie, Xin Sun, Xin Tong, and Derek Nowrouzezahrai. Hierarchical diffusion curves for accurate automatic image vectorization. *ACM Transactions on Graphics*, 33(6):1–11, Nov 2014.
26. Wen Dai, Tao Luo, and Jianbing Shen. Automatic image vectorization using superpixels and random walkers. In *2013 6th International Congress on Image and Signal Processing (CISP)*, Dec 2013.
27. Adobe Illustrator. https://www.adobe.com/products/illustrator.html.
28. Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
29. Mike Schuster and Kuldip K Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681, 1997.
30. Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
31. Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
32. Juan A Rodriguez, Abhay Puri, Shubham Agarwal, Issam H Laradji, Pau Rodriguez, Sai Rajeswar, David Vazquez, Christopher Pal, and Marco Pedersoli. StarVector: Generating scalable vector graphics code from images and text. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 16175–16186, 2025.
33. Yiying Yang, Wei Cheng, Sijin Chen, Xianfang Zeng, Fukun Yin, Jiaxu Zhang, Liao Wang, Gang Yu, Xingjun Ma, and Yu-Gang Jiang. OmniSVG: A unified scalable vector graphics generation model. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
34. Ronghuan Wu, Wanchao Su, and Jing Liao. LayerPeeler: Autoregressive peeling for layer-wise image vectorization. *arXiv preprint arXiv:2505.23740*, 2025.
35. Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
36. Ajay Jain, Amber Xie, and Pieter Abbeel. VectorFusion: Text-to-SVG by abstracting pixel-based diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1911–1920, 2023.
37. Vikas Thamizharasan, Difan Liu, Matthew Fisher, Nanxuan Zhao, Evangelos Kalogerakis, and Michal Lukac. Nivel: Neural implicit vector layers for text-to-vector generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4589–4597, 2024.
38. Ximing Xing, Haitao Zhou, Chuang Wang, Jing Zhang, Dong Xu, and Qian Yu. SVGDreamer: Text guided SVG generation with diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4546–4555, 2024.
39. Peiying Zhang, Nanxuan Zhao, and Jing Liao. Text-to-vector generation with neural path representation. *ACM Transactions on Graphics (TOG)*, 43(4):1–13, 2024.
40. Ximing Xing, Qian Yu, Chuang Wang, Haitao Zhou, Jing Zhang, and Dong Xu. Svgdreamer++: Advancing editability and diversity in text-guided svg generation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.
41. Zhengyi Wang, Cheng Lu, Yikai Wang, Fan Bao, Chongxuan Li, Hang Su, and Jun Zhu. ProlificDreamer: High-fidelity and diverse text-to-3d generation with variational score distillation. *Advances in Neural Information Processing Systems*, 36:8406–8441, 2023.
42. Ximing Xing, Chuang Wang, Haitao Zhou, Jing Zhang, Qian Yu, and Dong Xu. DiffSketcher: Text guided vector sketch synthesis through latent diffusion models. *Advances in Neural Information Processing Systems*, 36:15869–15889, 2023.
43. Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.
44. R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and Sabine Süsstrunk. SLIC superpixels compared to state-of-the-art superpixel methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, page 2274–2282, Nov 2012.
45. Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16×16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
46. Marco Cuturi and Mathieu Blondel. Soft-DTW: a differentiable loss function for time-series. In *International conference on machine learning*, pages 894–903. PMLR, 2017.
47. Shengming Yin, Zekai Zhang, Zecheng Tang, Kaiyuan Gao, Xiao Xu, Kun Yan, Jiahao Li, Yilei Chen, Yuxiang Chen, Heung-Yeung Shum, Lionel M. Ni, Jingren Zhou, Junyang Lin, and Chenfei Wu. Qwen-image-layered: Towards inherent editability via layer decomposition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16196–16205, 2026.
48. Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.
49. VisionCortex. Vtracer.

50. Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.
51. Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 586–595, 2018.